

Vector Store Backend Evaluation Report

ChromaDB vs pgvector for the TPI Retrieval Pipeline

Addressed to: TPI Centre
Prepared by: JSON Derulo Comeback Tour | DS205 | 1 June 2026

Executive Summary

This report summarizes our group’s evaluation of two vector store backends for TPI’s retrieval pipeline: ChromaDB and pgvector. The work was split across four members: pipeline infrastructure, the pgvector backend, benchmarking, as well as this report. The data below comes from a full benchmark run completed on 31 May 2026 against a corpus of 5,913 chunks from 17 PDFs across 15 companies.

Based on our available data, we recommend ChromaDB as the default. Retrieval quality is identical between the two backends, they returned the same top-5 results on every one of our 20 reference queries. Therefore, the decision comes down to speed and setup complexity, where ChromaDB has a clear advantage for the kind of day-to-day use TPI described.

Pgvector is faster at ingesting large batches of documents, which matters given that TPI does bulk uploads several times a year. But the difference at current corpus sizes is modest, and the added setup overhead (Docker, Postgres, Alembic migrations) makes it a harder default for a small research team. The recommendation might change if TPI moves to a production Postgres environment or if batch upload speed becomes the main bottleneck.

Dimension	Winner	Key Numbers
Retrieval quality	Tie	Identical recall@5, MRR, and precision@5 across all 20 queries
Query latency	ChromaDB	3.93 ms avg vs 8.97 ms avg; 15x faster on unfiltered queries
Ingestion throughput	pgvector	179.9 vs 157.8 chunks/sec total; 59% faster on store-only
Deployment complexity	ChromaDB	2 setup steps vs 5; no Docker required
Code legibility	ChromaDB	110 lines / 5 imports vs 174 lines / 11 imports
Metadata filtering	ChromaDB	Faster across company, sector, and year filter modes

1. Retrieval Quality

Before looking at speed, we wanted to confirm that both backends actually return the same results; there's no point recommending a faster backend if it retrieves worse chunks. We built a reference set of 20 questions manually, by reading through the source PDFs and identifying which chunks genuinely contain the answer to each question. Labels were assigned before running any benchmark so the results couldn't influence what we marked as correct.

Metric	ChromaDB	pgvector
Mean Recall@5	0.427	0.427
Mean Precision@5	0.180	0.180
Mean MRR	0.412	0.412
Same top-5 order	20 / 20	20 / 20

The two backends are completely equivalent on retrieval. This is expected as both store the same vector embeddings and use cosine similarity, but it's worth confirming empirically rather than assuming. It also means the rest of the evaluation is purely about speed, complexity, and fit.

One caveat worth flagging: a recall@5 of 0.427 means on average fewer than half the relevant chunks are appearing in the top 5 results. Both backends have the same score, so this isn't a backend problem, it reflects the difficulty of the retrieval task and the size of our reference set (20 queries across a 5,913-chunk corpus). It's something worth revisiting independently of the backend choice.

2. Query Latency

We measured query latency across four filter modes: unfiltered, filtered by company, by sector, and by year, running each of the 20 reference queries three times and taking the median. This gave us 60 data points per condition.

Backend	Filter Mode	Median (ms)	p95 (ms)
ChromaDB	Unfiltered	1.01	1.32
pgvector	Unfiltered	15.58	21.23
ChromaDB	Company	5.14	5.71
pgvector	Company	6.67	9.32
ChromaDB	Sector	4.28	5.65
pgvector	Sector	7.35	11.15
ChromaDB	Year	5.30	5.71

Backend	Filter Mode	Median (ms)	p95 (ms)
pgvector	Year	6.26	9.42

ChromaDB is faster in every filter mode. The biggest gap is on unfiltered queries, where ChromaDB is about 15x faster (1.01 ms vs 15.58 ms). Averaged across all four modes, ChromaDB’s median latency is 3.93 ms vs 8.97 ms for pgvector which is roughly 56% lower.

The reason for pgvector’s unfiltered penalty is the Docker overhead Sylvan mentioned in our meeting: every pgvector query goes through a containerised Postgres instance, which adds inter-process communication that ChromaDB avoids by running directly in-process. In absolute terms these are still very small numbers, sub-20ms either way, so for 25 daily users running short queries this won’t be noticeable. But the directional advantage is consistent across every condition we tested.

3. Ingestion Throughput

TPI already processes back catalog uploads of up to 2,000–3,000 documents at a time, and pgvector is meaningfully faster at that scale, making it the stronger choice on this dimension if ingestion becomes the priority.

Full Corpus

Backend	Chunks	Total (sec)	Store-only (sec)	Total chunks/sec	Store-only chunks/sec
ChromaDB	5,913	37.47	4.47	157.8	1,324
pgvector	5,913	32.86	2.82	179.9	2,100

Pgvector ingests the full corpus about 12% faster overall, and 59% faster on the store-only operation. Most of the total time is actually embedding (same model on both backends), so the store-only numbers give a cleaner picture of the backend difference. pgvector’s bulk updates and inserts into Postgres is simply more efficient than ChromaDB’s storage mechanism at this scale.

By Document Size

Page Group	Backend	Chunks	Total chunks/sec	Store-only chunks/sec
Below 60 pages	ChromaDB	916	152.5	1,464
Below 60 pages	pgvector	916	222.7	1,915
60 pages or more	ChromaDB	4,997	156.8	1,283
60 pages or more	pgvector	4,997	172.7	1,817

Pgvector's ingestion advantage holds regardless of document size: 46% faster chunks/sec for short documents, 14% faster for long ones. The gap is larger for shorter documents, which suggests pgvector's efficiency gains are most pronounced when processing many smaller files, which is probably the more common case for routine corpus updates.

To put this in context: at current corpus size (5,913 chunks), the total time difference is about 5 seconds. That's not a practical problem today. But if TPI scales to the 2,000–3,000 document batch uploads Sylvan described, the gap compounds significantly and pgvector becomes the stronger choice on this dimension alone.

4. Deployment Complexity

Backend	Extra Services	Docker Compose Lines	Clean-machine Setup Steps
ChromaDB	0	0	2
pgvector	1 (PostgreSQL)	26	5

ChromaDB installs as a Python package and works immediately. Pgvector needs Docker Desktop running, a Postgres container started, and an Alembic migration run before you can do anything, requiring five steps vs two. Sylvan also mentioned during our meeting that Windows setups can require additional workarounds for Docker, which adds another layer of friction for a team that may not have everyone on Mac or Linux.

This is probably the most underrated dimension in the comparison. The numbers make it look like a minor difference, but in practice “five setup steps” for a researcher who hasn't used Docker before is a meaningful barrier. ChromaDB just works more simply.

5. Code Complexity

Backend	File	Lines	Imports	Max Radon Score	Mean Radon Score
ChromaDB	vector_store/chroma.py	110	5	4	2.14
pgvector	vector_store/pgvector.py	174	11	6	2.50

Neither backend is particularly complex since radon complexity scores are low for both. But the ChromaDB implementation is 58% shorter and uses less than half the imports. Radon measures branching and nesting in code; a higher score means harder to read and maintain. The gap here isn't dramatic, but it's consistent with everything else: ChromaDB is the simpler option across the board.

This matters more over time than it does right now. If someone new joins the project in a year and needs to understand or modify the vector store layer, the ChromaDB implementation is the easier starting point.

6. Metadata Filtering

Both backends support filtering by company, sector, and year at query time. The pipeline exposes the same filtering interface regardless of which backend is running. ChromaDB and pgvector handle the implementation differently, but from the outside they behave identically and both return correct results.

It is worth noting that this metadata is not automatically extracted from the PDFs. It was manually added during ingestion, meaning any new document added to the pipeline requires its metadata to be explicitly tagged upfront. Filtering is only as reliable as the consistency of that tagging process.

ChromaDB is faster on filtered queries in our benchmark (company: 5.14 ms vs 6.67 ms, sector: 4.28 ms vs 7.35 ms, year: 5.30 ms vs 6.26 ms). That advantage holds where metadata is clean and consistently applied. Where data is messier or needs to be drawn from multiple sources, PostgreSQL's querying capabilities make it the more practical choice.

The more significant long-term consideration is whether TPI will need to filter using data that lives entirely outside the pipeline. For example, a separate database table tracking which companies belong to which investment portfolio. ChromaDB cannot perform that kind of lookup across systems. However Pgvector can, since it runs inside PostgreSQL and can combine vector search with any other table in the same database naturally. That capability is not required based on TPI's current setup, but it represents the most likely scenario in which pgvector becomes the stronger choice.

7. Recommendation

We recommend ChromaDB as the default backend for TPI's retrieval pipeline.

The retrieval quality is identical between the two backends, query latency is lower, and setup is simpler. For 25 daily users running short queries, ChromaDB is the faster and more straightforward option.

The one dimension where pgvector wins is ingestion throughput, which Sylvan flagged as TPI's real operational pain point. TPI already processes back catalog uploads of up to 2,000-3,000 documents at a time, and pgvector is meaningfully faster at that scale. If ingestion speed becomes the dominant priority over query speed, pgvector is the stronger choice.

The recommendation would flip toward pgvector under any of the following conditions:

- TPI moves to a production PostgreSQL deployment. If pgvector is running on an existing Postgres instance rather than a local Docker container, the latency overhead disappears and the ingestion advantage is retained with no extra complexity cost. At that point the case for pgvector becomes straightforward.
- Bulk ingestion becomes the dominant bottleneck. TPI already processes back catalog uploads of up to 2,000–3,000 documents at a time and pgvector is meaningfully faster at that scale. If the frequency of those uploads increases or the corpus grows significantly, pgvector's throughput advantage starts to matter operationally.
- Relational joins with structured metadata are required. If TPI needs to combine vector retrieval with company-level data stored in relational tables (sector classifications, entity metadata, etc.), pgvector inside Postgres can do that in a single query. ChromaDB cannot.

Appendix: How to Reproduce the Results

All benchmark results were produced from a corpus of 15 companies, 17 PDFs, and 5,913 chunks. Both backends were populated identically before benchmarking. The reference answer set of 20 queries was labelled manually before any benchmark run. Full setup instructions are in CONTRIBUTING.md.

Command	Purpose
<code>docker compose up -d && alembic upgrade head</code>	Start the pgvector backend
<code>python -m pytest</code>	Run correctness tests against both backends
<code>HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 python benchmark_metrics.py</code>	Run the full benchmark suite